

Tuần 8 — Deployment — CI/CD + CloudFormation/SAM + Elastic Beanstalk + ECS/ECR → HẾT Domain 3

Domain: Domain 3 – Deployment (24%) · **Thời lượng:** ~11h (4 buổi) · **Vị trí:** Tuần 1/1 — TRỌN Domain 3, có CHECKPOINT. Tuần nặng nhất.

Điều hướng:  [Tuần 7](#) ·  [Kế hoạch tổng](#) · [Tuần 9](#) 

 **CẢNH BÁO KHỐI LƯỢNG:** Tuần này gánh **TRỌN Domain 3 (24%)** trong 1 tuần → nặng nhất lộ trình. Nếu tràn giờ, **mượn thời gian từ Tuần 9** (Domain 4 nhẹ hơn & phần lớn kiến thức monitoring/troubleshooting bạn đã quen). Đừng để tuần này cắt lab.

Mục tiêu tuần này

- **Vẽ được** pipeline **CodeCommit** → **CodeBuild** → **CodeDeploy** → **CodePipeline** và nói rõ mỗi dịch vụ làm gì.
- **Tự tay** viết **buildspec.yml** chạy CodeBuild và **appspec.yml** deploy bằng CodeDeploy (In-place EC2 + Canary Lambda).
- **Phân biệt được** In-place vs Blue/Green và chọn đúng deployment config cho EC2 vs Lambda/ECS.
- **Viết được** template **CloudFormation** có **Parameters/Ref/GetAtt/Sub/Outputs**, tạo **change set**, và chia sẻ giá trị cross-stack bằng **Export/Fn::ImportValue**.
- **Chọn đúng** deployment policy của **Elastic Beanstalk** theo yêu cầu (zero-downtime, không dựng instance cũ...).
- **Giải thích được** task role vs execution role trong **ECS**, và luồng login/push/pull image lên **ECR**.
- **Chốt Domain 3:** đạt **≥70%** ở MINI-MOCK Domain 3 (~25 câu) trước khi sang Tuần 9.

Nội dung học chi tiết

A Buổi A — Lý thuyết (~3h): Developer Tools + Deployment strategies

1. Bộ Developer Tools — luồng CI/CD

- **CodeCommit**: Git repo có quản lý — từng bị **đóng nhận khách mới (7/2024)** rồi **mở lại GA (11/2025)**; để vẫn có thể hỏi như Source repo. Trigger được sự kiện.
- **CodeBuild**: build & test. Đọc **buildspec.yml** (mặc định ở **ROOT repo**), xuất **artifacts**, ghi log ra **CloudWatch Logs**.
- **CodeDeploy**: triển khai artifact lên **EC2/on-prem, Lambda, ECS**. Đọc **appspec.yml**.
- **CodePipeline**: **orchestration** — nối các **stage** (Source → Build → Deploy, có thể chèn **Approval** thủ công).
- Khái niệm nhắc qua: **CodeArtifact** (kho package/dependency riêng — npm/pip/maven), **CodeGuru** (review code bằng ML + profiler hiệu năng).
- **AWS AppConfig** — quản lý **feature flags** & cấu hình runtime, validate + triển khai từ từ (gradual rollout) + rollback tự động khi alarm; tách config khỏi code (bật/tắt tính năng ở test/prod env mà không deploy lại).
- **AWS Amplify**: hosting + CI/CD cho web/mobile front-end, tích hợp **Cognito/AppSync**; hợp cho full-stack serverless nhanh.

2. buildspec.yml (thuộc CodeBuild) — bắt buộc thuộc

- Các **phase chạy theo thứ tự**: **install** → **pre_build** → **build** → **post_build**.
- Các section chính: **env** (biến môi trường), **phases**, **artifacts** (file kết xuất), **cache** (giữ dependency giữa các lần build), **reports** (test report).
- **env** lấy secret 3 kiểu: **variables** (plaintext), **parameter-store** (SSM **Parameter Store**), **secrets-manager** (**Secrets Manager**).
- **ĐẶT Ở ROOT** repo; muốn đổi tên/vị trí phải khai báo trong project.

```
version: 0.2
env:
  parameter-store:
    DB_PASSWORD: /myapp/db/password      # kéo từ SSM Parameter Store
  secrets-manager:
    API_KEY: prod/apikey:key            # kéo từ Secrets Manager
phases:
  install:
    runtime-versions: { nodejs: 18 }
  pre_build:
    commands: [ "npm ci" ]
```

```

build:
  commands: [ "npm run build" ]
post_build:
  commands: [ "echo done" ]
artifacts:
  files: [ "**/*" ]
  base-directory: dist
cache:
  paths: [ "node_modules/**/*" ]

```

3. **appspec.yml** (thuộc **CodeDeploy**) — khác nhau theo target

- **EC2/on-prem:** có **files** (copy file) + **hooks** chạy đúng thứ tự lifecycle: **ApplicationStop** → **BeforeInstall** → **AfterInstall** → **ApplicationStart** → **ValidateService**.
- **Lambda:** khai báo **version/alias** + **traffic shift**; hooks: **BeforeAllowTraffic**, **AfterAllowTraffic**.
- **ECS:** hooks: **BeforeInstall**, **AfterInstall**, **AfterAllowTestTraffic**, **BeforeAllowTraffic**, **AfterAllowTraffic**.

4. Deployment strategies (RẤT hay hỏi)

- **In-place:** cập nhật ngay trên instance hiện có (dừng app → cài mới → khởi động). **CHỈ cho EC2/on-prem**. Rẻ nhưng có downtime/half-updated.
- **Blue/Green:** dựng môi trường mới song song rồi chuyển traffic. Rollback nhanh (quay lại "blue"). Áp dụng cho **EC2, ECS, Lambda**.
- **Deployment config theo target:**

Target	Các config CodeDeploy
EC2/on-prem	AllAtOnce / HalfAtATime / OneAtATime
Lambda & ECS	Canary (vd Canary10Percent5Minutes) / Linear (vd Linear10PercentEvery1Minute) / AllAtOnce

- **Canary** = shift 1 cục nhỏ rồi phần còn lại; **Linear** = shift đều theo bước; **AllAtOnce** = chuyển hết ngay.

B Buổi B — Hands-on (~3.5h): **buildspec** → **appspec** → **pipeline**

Lab 1 — Viết **buildspec.yml** + chạy CodeBuild

1. Đặt file **buildspec.yml** (mẫu ở Buổi A) tại **ROOT** repo.
2. Tạo CodeBuild project: source = repo, environment = managed image, **service role** đủ quyền đọc **SSM/Secrets Manager** + ghi **S3/Logs**.
3. Start build → xem log 4 phase chạy đúng thứ tự **install** → **pre_build** → **build** → **post_build**.
4. Kiểm tra **artifact** đã lên **S3**; bật **cache** để lần build sau nhanh hơn.

Lab 2 — **appspec.yml** + CodeDeploy (In-place trên EC2)

1. EC2 cài sẵn **CodeDeploy agent**; gán IAM role cho instance.
2. **appspec.yml** (In-place EC2):

```
version: 0.0
os: linux
files:
  - source: /
    destination: /var/www/app
hooks:
  ApplicationStop: [ { location: scripts/stop.sh } ]
  BeforeInstall: [ { location: scripts/before.sh } ]
  AfterInstall: [ { location: scripts/after.sh } ]
  ApplicationStart: [ { location: scripts/start.sh } ]
  ValidateService: [ { location: scripts/validate.sh } ]
```

3. Tạo CodeDeploy application + deployment group (deployment config **OneAtATime**), deploy → quan sát hooks chạy đúng thứ tự lifecycle.

Lab 3 — CodeDeploy Canary cho **Lambda**

1. **appspec.yml** (Lambda) trỏ tới function + alias, hooks **BeforeAllowTraffic/AfterAllowTraffic**.
2. Chọn config **Canary10Percent5Minutes** → 10% traffic sang version mới, sau 5 phút chuyển nốt.
3. Cho hook validation fail → xác nhận **rollback tự động** về version cũ.

Lab 4 — Ráp **CodePipeline** Source → Build → Deploy

1. Stage **Source**: **CodeCommit** (hoặc GitHub qua **CodeConnections** — trước là **CodeStar Connections**, đổi tên 2024) → thay đổi commit trigger qua **EventBridge**/webhook.

2. Stage **Build**: gọi CodeBuild (Lab 1).
3. Stage **Deploy**: gọi CodeDeploy (Lab 2). Artifact giữa các stage lưu ở **artifact store S3**.
4. (Tuỳ) chèn stage **Approval** thủ công trước Deploy. Push 1 commit → xem pipeline chạy hết chuỗi.

C Buổi C — Bổ sung (~2.5h): **CloudFormation + SAM + Beanstalk + ECS/ECR**

1. **CloudFormation** (IaC — trọng tâm Domain 3)

- **Template sections**: chỉ **Resources** là **BẮT BUỘC**; còn lại tùy chọn: **Parameters, Mappings, Conditions, Outputs, Transform, Metadata, Rules**.
- **Intrinsic functions**: **Ref, Fn::GetAtt, Fn::Sub, Fn::Join, Fn::Select, Fn::Split, Fn::FindInMap, Fn::ImportValue, Fn::If, Fn::Base64, Fn::GetAZs**.
- **Pseudo parameters**: **AWS::Region, AWS::AccountId, AWS::StackName**.
- **Change set**: **xem trước** thay đổi trước khi apply → tránh sửa/xoá ngoài ý muốn.
- **Nested stacks**: tách stack con tái dùng; **Cross-stack**: **Outputs** có **Export** + **Fn::ImportValue** để **dùng lại** giá trị ở stack khác.
- **DeletionPolicy**: **Delete** (mặc định) / **Retain** (giữ resource khi xoá stack) / **Snapshot** (chụp trước khi xoá — cho RDS/EBS...).
- **DependsOn**: ép thứ tự tạo resource. **Drift detection**: phát hiện resource bị sửa tay lệch template.

2. **SAM** (Serverless Application Model)

- Header **Transform**: **AWS::Serverless-2016-10-31** → CloudFormation "nở" cú pháp rút gọn.
- Resource rút gọn: **AWS::Serverless::Function / ::Api / ::HttpApi / ::SimpleTable / ::StateMachine**.
- Lệnh: **sam init** → **sam build** → **sam deploy --guided**; test local: **sam local invoke, sam local start-api**; đồng bộ nhanh: **sam sync**.
- **DeploymentPreference** trên function → shift traffic (Canary/Linear) **qua CodeDeploy** (an toàn khi update Lambda).

3. Elastic Beanstalk — PaaS

- **Deployment policies:**

Policy	Đặc điểm
All at once	nhanh, có downtime
Rolling	deploy theo lô, giảm capacity tạm thời
Rolling with additional batch	thêm lô mới để giữ đủ capacity (không giảm)
Immutable	tạo instance MỚI hoàn toàn, an toàn nhất, rollback dễ
Traffic splitting	canary — chia % traffic sang instance mới (cần ALB), giữ full capacity

⚠ **Blue/Green KHÔNG phải deployment policy** của Beanstalk — là kỹ thuật **swap CNAME/URL** giữa 2 environment (zero-downtime, rollback bằng swap ngược).

- Platform **Amazon Linux 2** đang bị retire dần, **Amazon Linux 2023** là nền tảng hiện hành.
- **.ebextensions/*.config** để tùy biến môi trường (package, option settings...).
- **Web tier** phục vụ HTTP; **Worker tier** đọc job từ **SQS** (xử lý nền).

4. ECS / Fargate + ECR

- **Task definition:** container, cpu/mem, IAM role. **Service:** desired count + gán **ALB**.
- **EC2 launch type** (tự quản instance) vs **Fargate** (serverless, không quản server).
- **TASK role** = quyền cho **app trong container** gọi AWS API. **EXECUTION role** = quyền **kéo image từ ECR + ghi log** (cho ECS agent). ★ Rất hay bấy.
- **ECR:** login rồi push/pull:

```
aws ecr get-login-password --region ap-southeast-1 | docker login --username AWS --password-stdin <acct>.dkr.ecr.ap-southeast-1.amazonaws.com
docker tag myapp:latest <acct>.dkr.ecr.ap-southeast-1.amazonaws.com/myapp:latest
docker push <acct>.dkr.ecr.ap-southeast-1.amazonaws.com/myapp:latest
```

lifecycle policy dọn image cũ; **image scanning** quét lỗ hổng.

Hands-on Buổi C:

1. Viết **CloudFormation** template có **Parameters** + **Ref/GetAtt/Sub** + **Outputs** (có **Export**); tạo **change set** rồi execute.
2. Tạo stack thứ 2 dùng **Fn::ImportValue** đọc giá trị Export của stack 1.
3. **sam init** → **sam build** → **sam local start-api** (test local) → **sam deploy --guided**.
4. **docker build** → **push image lên ECR** theo lệnh trên; bật scan + lifecycle policy.

D Buổi D — Practice + Review (~2h)

 **Bộ câu hỏi luyện tập của tuần:** [questions.md](#) — đáp án & giải thích: [answers.md](#). (bằng tiếng Anh — văn phong để thật để làm quen để.)

- Làm bộ câu hỏi CI/CD + **CloudFormation/SAM** + **Beanstalk** + **ECS/ECR**.
- ★ **MINI-MOCK Domain 3 (~25 câu)** — trộn toàn bộ chủ đề tuần. **Ghi số câu sai**, phân loại theo dịch vụ.
- **Spaced repetition:** ôn flashcard theo mốc **1 / 3 / 7 ngày** (thứ tự hooks appspec + tên deployment config rất dễ quên).
- Chỉ sang Tuần 9 khi mini-mock **≥70%**.

PHẢI NHỚ tuần này

Fact	Ghi nhớ
buildspec.yml	dùng bởi CodeBuild , nằm ở ROOT repo ; phase install → pre_build → build → post_build ; có env/artifacts/cache/reports
buildspec lấy secret	variables / parameter-store (SSM) / secrets-manager
appspec.yml	dùng bởi CodeDeploy
Hooks EC2/on-prem	ApplicationStop → BeforeInstall → AfterInstall → ApplicationStart → ValidateService

Fact	Ghi nhớ
Hooks Lambda	BeforeAllowTraffic , AfterAllowTraffic
Hooks ECS	BeforeInstall , AfterInstall , AfterAllowTestTraffic , BeforeAllowTraffic , AfterAllowTraffic
Config EC2	AllAtOnce / HalfAtATime / OneAtATime
Config Lambda & ECS	Canary / Linear / AllAtOnce
In-place	Chỉ EC2/on-prem
Blue/Green	EC2, ECS , Lambda
CodePipeline	stage Source/Build/Deploy/Approval; artifact ở S3 ; trigger EventBridge /webhook; GitHub qua CodeConnections (trước là CodeStar Connections)
CloudFormation bắt buộc	chỉ **Resources**
Pseudo params	AWS::Region , AWS::AccountId , AWS::StackName
Cross-stack	Outputs + Export + Fn::ImportValue
DeletionPolicy	Delete (mặc định) / Retain / Snapshot
SAM Transform	AWS::Serverless-2016-10-31
Beanstalk zero-downtime + không dừng instance cũ	Immutable
ECS role	task role = quyền app; execution role = kéo image ECR + ghi log

Bẫy đề hay gặp

- Thấy **buildspec.yml** hỏi thuộc dịch vụ nào → nhằm CodeDeploy → đúng là **CodeBuild** (và đặt ở **ROOT**).
- Thấy **appspec.yml** → nhằm CodeBuild/CodePipeline → đúng là **CodeDeploy**.
- Thấy "muốn **zero-downtime** + **KHÔNG dừng instance cũ**" ở Beanstalk → nhằm Rolling → đúng là **Immutable** (tạo instance mới hoàn toàn).

- Thấy "In-place cho Lambda/ECS" → **SAI**: In-place **CHỈ** EC2/on-prem; Lambda/ECS dùng **Blue/Green + Canary/Linear**.
- Thấy config **HalfAtATime/OneAtATime** áp cho Lambda → **SAI**: đó là **EC2**; Lambda/ECS là **Canary/Linear/AllAtOnce**.
- Nhầm **task role** với **execution role**: kéo image **ECR** không được / thiếu quyền log → thiếu **execution role**, không phải task role.
- Muốn chia sẻ giá trị giữa 2 stack mà lồng resource → đúng là **Export + Fn::ImportValue** (cross-stack), không phải copy-paste.
- Thấy "xem trước thay đổi trước khi apply CloudFormation" → là **change set**, không phải drift.
- Thấy "giữ lại DB/resource khi xóa stack" → **DeletionPolicy: Retain** (hoặc **Snapshot** nếu cần backup).
- Thấy secret trong build (DB password/API key) → **không hardcode**, dùng **parameter-store/secrets-manager** trong **env**.



Phản xạ nhanh (keyword → đáp án)

Thấy từ khóa	Bật ngay
file mô tả build, phase install/build	buildspec.yml (CodeBuild, ROOT repo)
file mô tả deploy, hooks lifecycle	appspec.yml (CodeDeploy)
orchestrate Source→Build→Deploy	CodePipeline
kho npm/pip/maven riêng	CodeArtifact
review code / profiler bằng ML	CodeGuru
zero-downtime + instance mới hoàn toàn (Beanstalk)	Immutable
giữ đủ capacity khi deploy (Beanstalk)	Rolling with additional batch
shift 10% rồi phần còn lại (Lambda/ECS)	Canary
shift đều theo bước (Lambda/ECS)	Linear
rollback nhanh, 2 môi trường song song	Blue/Green
xem trước thay đổi CloudFormation	change set

Thấy từ khoá	Bật ngay
chia sẻ giá trị giữa các stack	Export + Fn::ImportValue
giữ resource khi xoá stack	DeletionPolicy: Retain
resource bị sửa tay lệch template	drift detection
cú pháp serverless rút gọn	SAM (Transform)
worker tier đọc job nền	SQS
kéo image ECR + ghi log cho ECS	execution role
quyền app trong container gọi AWS	task role



Lab checklist

- ☐ Viết **buildspec.yml** ở ROOT + chạy CodeBuild thành công, thấy artifact + cache.
- ☐ Viết **appspec.yml** EC2 + deploy In-place, hooks chạy đúng thứ tự lifecycle.
- ☐ Deploy Lambda bằng CodeDeploy **Canary10Percent5Minutes**, thử rollback.
- ☐ Ráp **CodePipeline** Source → Build → Deploy (có Approval).
- ☐ Viết template **CloudFormation (Parameters/Ref/GetAtt/Sub/Outputs)** + tạo change set.
- ☐ Export ở stack 1, **Fn::ImportValue** ở stack 2.
- ☐ **sam init/build/local start-api/deploy --guided**.
- ☐ Build & push image lên **ECR** (login + tag + push), bật scan + lifecycle.



Cổng tự kiểm tra (phải trả lời trôi chảy mới sang tuần sau)

- buildspec.yml** thuộc dịch vụ nào & đặt ở đâu? Đáp án gọn: **CodeBuild**; đặt ở **ROOT** repo. Phase: **install** → **pre_build** → **build** → **post_build**.
- appspec.yml** thuộc dịch vụ nào? Đáp án gọn: **CodeDeploy**. EC2 có **files** + hooks lifecycle; Lambda/ECS khai báo traffic shift + hooks AllowTraffic.
- Beanstalk muốn zero-downtime + KHÔNG dừng instance cũ** → **policy** nào? Đáp án gọn: **Immutable** (tạo instance mới hoàn toàn, rollback dễ). (Blue/Green

cũng zero-downtime nhưng là **kỹ thuật swap CNAME**, KHÔNG phải deployment policy.)

- **In-place khác Blue/Green thế nào? Đáp án gọn:** In-place cập nhật trên instance cũ (chỉ EC2/on-prem, có downtime); Blue/Green dựng môi trường mới rồi chuyển traffic (EC2/ECS/Lambda, rollback nhanh).
- **Cross-stack chia sẻ giá trị dùng cặp hàm nào? Đáp án gọn:** stack nguồn khai **Outputs** + **Export**; stack đích đọc bằng **Fn::ImportValue**.
- **Task role vs execution role trong ECS? Đáp án gọn:** task role = quyền cho app trong container; execution role = kéo image **ECR** + ghi log.
- ★ **CHECKPOINT Domain 3:** đã đạt **≥70%** ở MINI-MOCK Domain 3 (~25 câu) chưa? Chưa → **KHÔNG** sang Tuần 9, ôn lại câu sai trước.



Tài nguyên tuần này

📁 **Đã crawl sẵn tài liệu AWS vào [resources/](#)** — đọc offline được.

- AWS Docs: **AWS CodeBuild** User Guide — build spec reference (**buildspec.yml**), phases, env/artifacts/cache.
- AWS Docs: **AWS CodeDeploy** User Guide — AppSpec file reference, lifecycle event hooks (EC2/Lambda/ECS), deployment configurations.
- AWS Docs: **AWS CodePipeline** User Guide — stages, artifacts, source integrations (CodeConnections).
- AWS Docs: **AWS CloudFormation** User Guide — template anatomy, intrinsic functions, change sets, cross-stack Export/ImportValue, **DeletionPolicy**, drift.
- AWS Docs: **AWS SAM** Developer Guide — template spec, **sam** CLI, **DeploymentPreference**.
- AWS Docs: **AWS Elastic Beanstalk** Developer Guide — deployment policies, **.ebextensions**, worker environments.
- AWS Docs: **Amazon ECS** Developer Guide (task/execution role, Fargate) + **Amazon ECR** User Guide (push/pull, lifecycle, scanning).
- Khoá học: Stephane Maarek — mục Developer Tools (CI/CD), CloudFormation, SAM, Beanstalk, ECS/ECR; Adrian Cantrill — deployment & IaC.



Checklist hoàn thành Tuần 8

- ☐ Hoàn thành 4 buổi A/B/C/D
- ☐ Thuộc thứ tự hooks **appspec.yml** (EC2/Lambda/ECS) + các deployment config

- ☐ Phân biệt In-place vs Blue/Green và chọn đúng config theo target
- ☐ Hoàn thành 8 mục Lab checklist (CI/CD + CloudFormation + SAM + ECR)
- ☐ Chọn đúng deployment policy Beanstalk theo yêu cầu
- ☐ **Đạt $\geq 70\%$ MINI-MOCK Domain 3 (~25 câu)** — CHECKPOINT (kết thúc Domain 3)
- ☐ Vượt Cổng tự kiểm tra